

The first question is whether we have sufficient data to train and test the model. Next, we need to ensure that the data we have is representative of the real world problem we are addressing. We also need to ensure that the data is free of biased/noisy labels, and that the training and test data are of the same distribution. Testing also needs to ensure that input data is free of data poisoning/adversarial input attacks. It is important to remember that data testing is a unique requirement for ML applications, and this does not happen in traditional software testing. For traditional software, the testing of program code happens with many sets of data. So the entity under test in traditional software is always the program code. On the other hand, in testing ML applications, we have to test data separately and the model code separately. When we test data, 'data' is the entity under test and the input can be a simple 'ML' model which is used to verify that the data is of good quality. For instance, if we want to identify whether data contains adversarial or poisoned inputs, we may use a simple logistic regression based classifier as the model to isolate this before using the data on our final model.

3. What are the real world deployment scenarios for which we need to test our model? Testing that needs to be performed for an ML model that is being deployed for email spam detection is quite different from the testing that is required for a self-driving car's control system. Hence, our testing of ML applications needs to factor in the scenario the ML application is being deployed in.
4. As in the case of traditional software testing, the testing workflow consists of test generation, test execution and test evaluation. Traditional software testing is simpler because the expected output can be determined in advance, so that test oracles are readily available to evaluate the test results. In the case of ML applications, we may not know in advance what the correct output is for a given test input (unless we have human annotators who have already labelled the inputs with the expected output). The lack of a readily available test oracle is a major problem when evaluating the test results of ML applications. We will discuss the alternatives and workarounds for this issue in subsequent columns.

So far, we have been talking mostly about offline testing of an ML application, which is typically done with historical data. Consider the example we discussed of the sentiment analysis of tweets. As historical data, you would have access to a large corpus of tweets, which are labelled with three sentiment category labels — positive, negative and neutral. This historical data can be split into training data, validation data and test data in order to train the model and test its performance. In addition to that, developers can set aside a part of the historical data for either test inputs/test case generation or to synthesise their own test inputs.

Online testing happens in the real application environment. For instance, let us assume that your corporate website wants to show the sentiment category of each tweet posted about your company in real-time. In that case, as the tweet gets posted onto Twitter, you will have a data stream coming into your application, which becomes the input to the sentiment prediction model. Testing the ML application in this real world use case is online testing, which also needs to factor in the possibility that some of the assumptions made with historical data may not hold true in real life deployments. A simple example of this could be that the historical corpus on which offline testing was done in the above example, consisted of tweets that did not contain any emoticons. Hence the system never needed to deal with emoticons interleaved in text. On the other hand, online testing will quickly reveal this issue.

Another key difference between traditional software testing and testing ML applications is about application behaviour. Standard software behaviour is NOT expected to change unless there is a code or configuration change. On the other hand, ML applications can exhibit considerable change in behaviour without any change in model code, framework code or configuration, and this is due to retraining the model on new data. Hence, any retraining of the model would necessitate re-testing to ensure the software quality.

Given that the computational resources for testing ML applications are quite expensive (since ML applications require considerable processing and memory), and that input space for ML applications is very large and high dimensional, testing all possible inputs and scenarios is not practical. Hence, test prioritisation and coverage, which are important in traditional software testing, become even more critical in testing ML applications. It is also NOT very clear what kind of testing would be needed to ensure satisfactory performance in various auxiliary properties of ML applications such as fairness, robustness and interpretability. Since ML is an evolving field, many of these properties do not have a computable or industry standard definition. Considering this ambiguity, testing for these properties becomes even more complicated. We will discuss this issue in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month! **END** 🐧

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.